TP noté : maîtriser Markdown et concevoir un README

BUT Informatique, 2e année, semestre 3 · R3.04 Qualité de développement · SAÉ 3.Real.01

Durée du TP	120 minutes (voir le découpage en deux séances plus bas)
Modalité	séance **notée sur 20**, seul ou en binôme
Rendu	lien de votre dépôt GitHub ; archive `.zip` de secours si GitHub est indisponible
Compétences visées	AC21.03 (bonnes pratiques de conception et de programmation), AC21.04 (vérifier et valider la qualité par les tests) ; en binôme, AC26.02 (intégrer une équipe)
Matériel	VS Code (aperçu Markdown intégré), Git, Python 3.10 ou plus récent, un compte GitHub

1. Contexte et objectifs

Un collègue vous confie `pluvio`, un petit outil Python en ligne de commande. Il lit un fichier CSV de relevés de pluie de communes de Martinique et calcule des statistiques. **Les données sont entièrement fictives** : elles ne proviennent d'aucune station de mesure. Le code fonctionne, mais le projet n'a pas de documentation. Votre mission : le rendre utilisable par quelqu'un qui ne l'a jamais vu.

À la fin du TP, vous saurez :

1. écrire un document Markdown valide (CommonMark et extensions GFM) ;
2. diagnostiquer et réparer des erreurs de rendu fréquentes ;
3. structurer le README d'un projet Python ;
4. documenter du code (docstrings, PEP 257) et le tester avec `pytest` ;
5. travailler avec Git et GitHub : branches, fusion, messages de commit, revue ;
6. évaluer un README avec une grille de critères.

Sont évalués : les tests automatiques, la structure et le contenu du README, le fonctionnement des exemples qu'il contient, la qualité des messages de commit, l'usage des branches et de la fusion, la revue réalisée avec la grille, et la contribution de chaque membre du binôme visible dans `git log`.

2. Organisation

2.1 Seul ou en binôme

Étape	Seul	En binôme (étudiants A et B)
Ex.0	vous créez votre dépôt à partir du modèle, puis le clonez	A crée le dépôt à partir du modèle (bouton « Use this template » ou lien fourni par l'enseignant) et ajoute B comme collaborateur ; les deux clonent
Ex.1	vous	A
Ex.2	vous	B (en parallèle d'A)

Étape	Seul	En binôme (étudiants A et B)
Point de contrôle	commit + push	chacun pousse son fichier (voir 2.3)
Ex.3	vous	A rédige titre, Description, Prérequis, Installation, Utilisation ; B rédige Tests, Arborescence, Contribution, Licence, Contact
Ex.4	vous	B (A relit les docstrings et lance les tests)
Ex.5	deux branches, auto-revue	chacun sa branche, revue croisée

En binôme, chaque étudiant configure son identité Git **avec son propre nom** (commandes en Q0.1, après le clone) : c'est elle qui apparaît dans l'historique.

2.2 Règles communes

- Un **message de commit** tient sur une ligne de 12 à 72 caractères et a la forme `type: résumé`, où `type` vaut `docs`, `fix`, `test`, `feat` ou `chore`. Exemple : `docs: ajouter la section Installation`. Les messages créés par Git ou GitHub pour une fusion (« Merge ... ») sont acceptés tels quels.
- Faites des commits **courts et fréquents** : un commit = une idée.
- N'effacez aucune branche avant le rendu et ne réécrivez pas l'historique (pas de `git push --force`).
- Ne modifiez ni `tests/` ni `pytest.ini`.
- Écrivez les fichiers en UTF-8 (réglage par défaut de VS Code).
- Nommez-vous dans l'historique : pas de commit anonyme ni sous un autre nom.

2.3 Découpage en deux séances

Le TP dure 120 minutes. Il peut se faire en une fois ou en deux séances de 90 minutes : la première commence par le cours (40 minutes) et se termine par un bilan de mi-parcours, la seconde finit par le rendu.

Séance	Contenu
1	Ex.0, Ex.1, Ex.2, puis **point de contrôle** : `git commit` et `git push`
2	`git pull`, puis Ex.3, Ex.4, Ex.5 et rendu

Le point de contrôle sert de sauvegarde : tout ce qui n'est pas poussé à la fin de la séance 1 n'existe pas pour la séance 2 (autre poste, autre session).

```
git add exercices
git commit -m "docs: terminer les exercices 1 et 2"
git pull --rebase
git push
```

En binôme, `git pull --rebase` avant `git push` évite le refus de push quand l'autre a poussé avant vous.

Ex.3 en binôme. Pour éviter les conflits dans `README.md`, A commence par pousser un squelette : le titre et les neuf titres de niveau 2 de C3, sans contenu. B fait `git pull`. Ensuite, chacun ne modifie que ses sections et fait `git pull --rebase` avant chaque `git push`.

Ex.0 : préparer l'environnement (10 min)

Q0.1 Obtenez votre dépôt : créez-le à partir du modèle indiqué par l'enseignant (ou suivez le lien d'invitation qu'il fournit), puis clonez-le et entrez dedans. Ensuite, configurez votre identité **dans ce dépôt**.

```
git clone ADRESSE_DE_VOTRE_DEPOT pluvio
cd pluvio
git config user.name "Prénom Nom"
git config user.email "adresse@exemple.fr"
```

Q0.2 Créez un environnement virtuel dans le dossier `.venv`, activez-le, puis installez les dépendances de `requirements.txt`.

Linux et macOS (bash) :

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Windows (PowerShell) :

```
py -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

Si PowerShell refuse d'exécuter le script d'activation, demandez de l'aide à l'enseignant plutôt que de modifier la politique d'exécution du poste.

Q0.3 Lancez les tests.

```
pytest
```

Plusieurs tests **échouent** : c'est normal. Repérez dans la sortie les deux fichiers de tests concernés. Les fichiers `tests/test_stats.py` et `tests/test_cli.py` doivent réussir.

Q0.4 Vérifiez que l'outil fonctionne et notez la sortie, dont vous aurez besoin pour l'Ex.3.

```
python -m pluvio.cli data/pluies_fictives.csv --par-commune
```

Ex.1 : chasse aux erreurs (15 min)

Le fichier `exercices/ex1_erreurs.md` contient **12 erreurs** de syntaxe Markdown. Chacune produit un rendu incorrect ou dégradé.

Q1.1 Ouvrez le fichier dans VS Code puis ouvrez l'aperçu : raccourci `Ctrl+Maj+V` (Windows, Linux) ou `Cmd+Maj+V` (macOS). Comparez le texte source et le rendu : repérez les anomalies visibles.

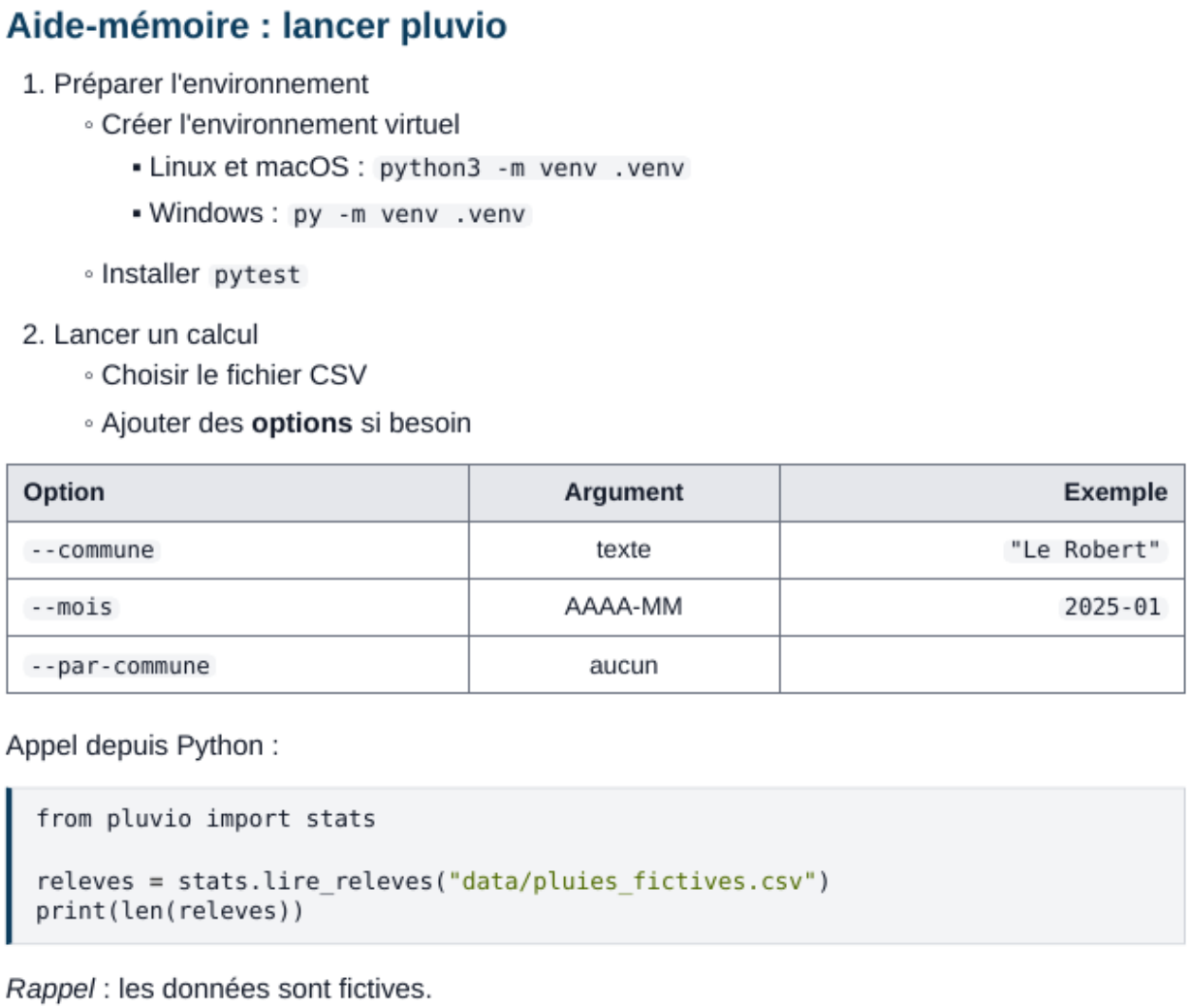
Q1.2 Corrigez le fichier **directement** (même nom, même emplacement) pour que le rendu soit celui qu'on attend d'une fiche d'utilisation soignée. Ne supprimez aucun contenu et n'ajoutez aucune section.

Q1.3 Vérifiez l'aperçu final, puis faites un commit.

Les erreurs ne se voient pas toutes de la même façon : certaines changent l'apparence, d'autres font disparaître du texte, d'autres sont invisibles à l'écran mais nuisent à l'accessibilité ou à la fiabilité du document.

Ex.2 : reproduire un rendu (10 min)

L'image `exercices/ex2_rendu_cible.png` montre un rendu Markdown (mise en page approximative : le rendu exact dépend du logiciel qui l'affiche).



Aide-mémoire : lancer pluvio

1. Préparer l'environnement
   - Créer l'environnement virtuel
     - Linux et macOS : `python3 -m venv .venv`
     - Windows : `py -m venv .venv`
   - Installer `pytest`
2. Lancer un calcul
   - Choisir le fichier CSV
   - Ajouter des **options** si besoin

Option	Argument	Exemple
`--commune`	texte	`"Le Robert"`
`--mois`	AAAA-MM	`2025-01`
`--par-commune`	aucun	

Appel depuis Python :

```
from pluvio import stats

releves = stats.lire_releves("data/pluies_fictives.csv")
print(len(releves))
```

Rappel : les données sont fictives.

Rendu cible de l'exercice 2

Q2.1 Éditez `exercices/ex2_rendu.md` pour que son aperçu reproduise la cible : mêmes titres, mêmes listes imbriquées, même tableau (y compris l'alignement des colonnes), même bloc de code Python avec le bon langage, même mise en forme du texte.

Q2.2 Le code Python du bloc doit fonctionner tel quel depuis la racine du dépôt. Vérifiez-le en le recopiant dans un terminal Python (`python`).

Q2.3 (point de contrôle) Faites un commit puis un `git push` (voir 2.3).

Ex.3 : rédiger le README (35 min)

Remplacez le contenu de `README.md`. Votre README doit satisfaire **tous** les critères suivants, que l'on peut vérifier un par un.

N°	Critère
C1	Un seul titre de niveau 1, en première ligne, égal au nom du projet : `pluvio`.
C2	Juste après le titre : une phrase ou une citation qui indique que les données sont **fictives**. La section Description (C3) dit en 1 à 5 lignes ce que fait l'outil et à qui il s'adresse.
C3	Neuf sections de niveau 2, dans cet ordre, avec ces intitulés exacts : Description, Prérequis, Installation, Utilisation, Tests, Arborescence, Contribution, Licence, Contact.
C4	Prérequis : liste à puces nommant Python (version minimale), Git et `pytest`.
C5	Installation : commandes pour bash **et** pour PowerShell, chacune dans un bloc de code de langage `bash` ou `powershell`.
C6	Utilisation : au moins deux commandes `python -m pluvio.cli ...` dont une avec un argument entre guillemets (nom de commune), chacune suivie de sa **sortie réelle** dans un bloc de langage `text`.
C7	Utilisation : un tableau Markdown décrivant les trois options de la commande (voir `python -m pluvio.cli --help`).
C8	Une image du fichier `docs/schema_flux.svg` avec un texte alternatif non vide.
C9	Tests : la commande à lancer et le résultat attendu, en une phrase.
C10	Arborescence : un bloc de langage `text` qui liste des fichiers **réels** du dépôt.
C11	Contribution : une liste numérotée d'au moins trois étapes.
C12	Licence : le nom de la licence et un lien relatif vers le fichier `LICENSE`.
C13	Contact : un moyen de contact **sans donnée personnelle** (pas de numéro de téléphone ni d'adresse personnelle).
C14	Tous les blocs de code ont un langage ; aucune commande ne commence par une invite (`$` ou `PS>`).
C15	Aucune ligne de plus de 80 caractères, comptés accents compris. Les blocs de code et les lignes de tableau sont exemptés ; titres, listes, citations et images sont comptés.

Deux remarques :

- Le test `tests/test_readme.py` rejoue les commandes de vos blocs `bash` et `powershell` qui lancent `python -m pluvio.cli` ou un script `.py`, et compare leur sortie à votre texte. Les commandes d'installation (`venv`, `pip`, `git`) et `pytest` ne sont pas rejouées. Une commande par ligne, sans `\` de continuation.
- Les commandes du README doivent s'exécuter **depuis la racine du dépôt**.

Faites un commit à chaque section terminée, puis lancez `pytest tests/test_readme.py` : le but est de le voir passer entièrement.

Ex.4 : compléter `verifier_readme.py` (20 min)

Le fichier `verifier_readme.py` vérifie un README. Deux fonctions sont à écrire. Les autres (`normaliser`, `titres`, `main`) sont fournies et fonctionnent.

Q4.1 `sections_manquantes(texte, attendues=SECTIONS_ATTENDUES)` renvoie la liste, dans l'ordre de `attendues`, des sections absentes. Une section est présente si **un titre de n'importe quel niveau contient son nom**, sans tenir compte de la casse ni des accents. Les titres situés dans un bloc de code ne comptent pas (la fonction `titres` s'en charge déjà).

Q4.2 `blocs_sans_langage(texte)` renvoie la liste des **numéros de ligne** (la première ligne est la ligne 1) qui ouvrent un bloc de code délimité par trois accents graves, sans langage. La ligne qui ferme un bloc ne compte pas.

Q4.3 Chaque fonction doit avoir une docstring conforme à la PEP 257 :

- le résumé tient sur **une ligne**, commence juste après les guillemets d'ouverture (règle de cette séance), se termine par un point et fait 79 caractères au plus, guillemets et indentation exclus ;
- si la docstring a une suite, une **ligne vide** sépare le résumé de la suite.

Q4.4 Lancez `pytest tests/test_verifier_readme.py` jusqu'à ce que tous les tests passent, puis contrôlez votre propre README :

```
python verifier_readme.py README.md
```

Corrigez ce que le rapport signale, puis faites un commit.

Ex.5 : branches, fusion et revue (20 min)

Cet exercice utilise la **grille de qualité** `grille_qualite_readme.md` et le modèle `docs/revue_modele.md`. Le but est de relire un README avec des critères observables, puis de fusionner proprement les améliorations.

La **fusion** (*merge*) réunit dans `main` les commits d'une branche. Une **pull request** (PR) est une demande de fusion publiée sur GitHub : on y relit les changements avant de fusionner.

Déroulé

Étape	En binôme	Seul
Q5.1	`main` à jour (`git switch main`, `git pull`). Chacun crée sa branche `revue/IDENTIFIANT`	`main` à jour, puis deux branches : `revue/IDENTIFIANT` et `amelioration/IDENTIFIANT`
Q5.2	Chacun remplit la grille pour **tout** le README, dans `revues/revue-IDENTIFIANT.md` (copie de `docs/revue_modele.md`), avec une preuve par critère	Vous remplissez la grille dans `revues/revue-IDENTIFIANT.md`, sur la branche `revue/...`
Q5.3	Chacun applique **une** amélioration trouvée en Q5.2 dans la partie **de l'autre**, sur sa branche	Vous appliquez au moins **trois** améliorations de votre liste sur la branche `amelioration/...`
Q5.4	Chacun pousse sa branche et ouvre une PR vers `main`	Vous poussez les deux branches et ouvrez deux PR
Q5.5	Chacun relit la PR de l'autre avec la grille et laisse au moins un commentaire utile	Vous relisez vos deux PR avec la grille et laissez un commentaire dans chacune
Q5.6	Chacun fusionne la PR de l'autre après revue, avec « Merge pull request » puis l'option « Create a merge commit » (libellés anglais ; pas de *squash*)	vous fusionnez vos deux PR après auto-revue, même option

Dans la grille, notez chaque critère de 0 à 2 en citant une preuve (numéro de ligne ou extrait). Terminez par trois améliorations concrètes.

Règles de la revue : un commentaire utile désigne une ligne ou un extrait, dit ce qui ne va pas **et** propose une reformulation.

Étape facultative Q5.7 : un conflit volontaire (5 min, dans les 20 min)

Si le temps le permet, ajoutez à l'étape Q5.3, **sur chaque branche** (seul : sur `revue/...` et sur `amelioration/...`), une ligne `Relu par : IDENTIFIANT` en **dernière ligne de la section Contact** du README. Fusionnez la première branche sur GitHub, puis récupérez `main` dans la seconde : Git signale un **conflit** (deux modifications différentes au même endroit). Résolvez-le sur votre poste, puis poussez.

```
git switch revue/IDENTIFIANT
git fetch origin
git merge origin/main
# éditer README.md : retirer les marqueurs <<<<<<<, =======, >>>>>>>
git add README.md
git commit -m "fix: résoudre le conflit sur la ligne Relu par"
git push
```

Quand le conflit est résolu, plus aucun marqueur ne doit rester dans le fichier. Cette étape est facultative et n'est pas notée.

Après la fusion

```
git switch main
git pull
pytest
```

Les tests de l'outil et ceux du README doivent réussir sur `main`.

Rendu (10 min)

1. Vérifiez que tout est poussé : `git status` ne doit rien signaler et `git log --oneline` doit correspondre à l'historique visible sur GitHub.
2. Déposez le **lien** de votre dépôt selon la consigne de l'enseignant, en indiquant le nom de chaque membre du binôme.
3. **Plan B** si GitHub est indisponible : créez l'archive de secours, puis déposez `rendu.zip` selon la consigne.

```
python rendre_zip.py
```

Le script crée `rendu.zip` à la racine du dépôt, sans l'environnement virtuel. L'archive contient le dossier `.git` : l'historique fait partie du rendu.

Sans GitHub, il n'y a pas de pull request : fusionnez vos branches en local avec `git merge --no-ff NOM_DE_BRANCHE` (l'option `--no-ff` force la création d'un commit de fusion).

Annexe : aide-mémoire Git

Besoin	Commande
Voir l'état	`git status`
Créer une branche et s'y placer	`git switch -c NOM`
Changer de branche	`git switch NOM`
Publier une nouvelle branche	`git push -u origin NOM`
Récupérer les changements distants	`git pull`
Fusionner une branche dans la branche courante	`git merge NOM`
Voir l'historique	`git log --oneline --graph --all`